DESAFIO — ENTREGA INICIAL:
ESPECIFICAÇÃO TÉCNICA E GOVERNANÇA DE CONTEXTO (SDD & AI HARNESS)

Disciplina:
Bootcamp III

Carga Horária:
75 horas

Modalidade:
EAD — Disciplina Virtual (DV)

Prazo de Entrega:
[Inserir Data no Cronograma do AVA]

Atividade:
Desafio — Entrega Inicial
(Atividade Cumulativa — Fase 1)

Tipo de Trabalho:
Individual

Valor Avaliativo:
10,0 pontos
(10% da pontuação total da disciplina | Cômputo de **7,5h** de frequência)

Forma de Submissão:
Envio do arquivo **PDF** do trabalho contendo obrigatoriamente o link do documento no **Google Docs** e o link do vídeo de apresentação gravado no **YouTube.**

CONTEXTUALIZAÇÃO E OBJETIVO GERAL

Seja bem-vindo(a) ao Bootcamp III! Neste módulo final, você experimentará o paradigma mais moderno da Engenharia de Software: o **Spec-Driven Development (SDD)** no desenvolvimento de sistemas assistidos por ferramentas de **Inteligência Artificial.**

Na era dos assistentes inteligentes (Coding Agents), o papel do desenvolvedor evoluiu de "escrever linhas de código manualmente" para **projetar especificações técnicas rigorosas, definir regras de contexto** e atuar como **homologador crítico.**

O objetivo deste Desafio é construir a documentação de especificação executável e governança de IA para o sistema **TaskAnalyzer** (módulo avançado de métricas e produtividade) utilizando o **Google Docs**, e apresentar o seu planejamento em um **vídeo explicativo de 5 minutos.**

Importante (Atividade Cumulativa):
Esta entrega corresponde à **Fase 1** de um projeto em **3 etapas.** A especificação técnica e as regras de contexto que você estruturar agora no **Google Docs** servirão como o contrato que orientará a geração automatizada de código no **GitHub** com assistentes de IA (**Fase 2**) e o pipeline de integração contínua e revisão ética (**Fase 3**)!

PROJETO CUMULATIVO — EM 3 ETAPAS

FASE 1
Esta entrega

Especificação e Governança (SDD & AI Harness)

- Entendimento do problema
- Especificação técnica no Google Docs (SDD)
- Regras de contexto e governança de IA
- Vídeo de apresentação (5 min)

Valor: 10,0 pts

FASE 2
Entrega Intermediária

Implementação com IA e Versionamento

- Criação do repositório no GitHub
- Uso de assistentes de IA para geração de código
- Implementação do sistema TaskAnalyzer
- Testes e versionamento

Valor: 15,0 pts

FASE 3
Entrega Final

DevOps, Testes e Revisão Ética

- Pipeline de integração contínua (CI/CD)
- Containerização (Docker)
- Testes automatizados
- Revisão ética e entrega final

Valor: 25,0 pts

DICA DE SUCESSO: Quanto mais **clara, estruturada e completa** for a sua especificação e as regras de contexto, melhor será a **qualidade do código** gerado pela IA e mais eficiente será o seu **papel como desenvolvedor moderno e estrategista de software!**

DETALHAMENTO DO CONTEÚDO OBRIGATÓRIO (GOOGLE DOCS)

Acesse o Google Docs, crie um documento intitulado `[Especificacao SDD TaskAnalyzer - [Seu Nome Completo]` e responda detalhadamente às seções abaixo:

1 VISÃO GERAL E CONTRATO DE NEGÓCIO (SDD)

Identificação:	Seu nome completo, curso, polo/turma e e-mail institucional.
Propósito do Módulo TaskAnalyzer:	Apresentação do escopo do sistema de análise de tarefas e produtividade (cálculo de tempo médio de conclusão, taxa de atraso e indicadores por prioridade).
Contrato Executável de Interface:	Detalhamento formal das entradas, saídas, tipos de dados e restrições de negócio que o código deverá seguir estritamente.

FOCO DESTA SEÇÃO

Definir o contrato executável que será a fonte da verdade para o código gerado por IA na próxima fase.



2 ESPECIFICAÇÃO DE CENÁRIOS DE ACEITE E TEST HARNESS

Cenários de Teste (Dado-Quando-Então):

Cenário 1 (Sucesso):
Dado um conjunto de tarefas válidas,
quando o analisador for executado,
então deve retornar as métricas corretas.

Cenário 2 (Exceção/Erro):
Dado uma entrada com datas inválidas ou divisão por zero,
quando o analisador for executado,
então deve disparar uma exceção específica tratada.

Planejamento do Test Harness:

Explicação de como esses cenários se transformarão em testes automatizados (pytest) na Fase 2 para garantir que o código gerado por IA esteja correto.



IMPORTANTE: Esta especificação é a base do desenvolvimento assistido por IA. Seja claro, objetivo e rigoroso nas definições!

3 GOVERNANÇA DE CONTEXTO E REGRAS PARA AGENTES DE IA (CONTEXT_RULES)

Regras rígidas que serão fornecidas ao assistente de IA para garantir alinhamento, consistência e qualidade do código gerado.

Diretrizes Arquiteturais (Obrigatórias)

- Python 3.11+ como versão mínima obrigatória.
- Utilizar **type hints** (anotações de tipo) em todas as funções e métodos.
- Seguir o princípio **Single Responsibility** (SRP) e código limpo (PEP 8).
- Documentação formal no padrão Google style docstrings para módulos, classes, funções e parâmetros.
- Funções devem ser pequenas, coesas e com um único propósito.
- Tratamento de exceções específico e com mensagens claras.
- Logs estruturados utilizando o módulo padrão **logging**.
- Código deve ser legível, manutenível e com nomes de variáveis e funções descritivos.

Proibições Explícitas (Regras Restritivas)

- Não utilizar bibliotecas externas não autorizadas.
- Não alterar, remover ou adaptar os **cenários de teste** fornecidos.
- Não modificar a estrutura de pastas definida nesta especificação.
- Não persistir dados em arquivos ou bancos de dados.
- Não alterar a assinatura (nome e parâmetros) das funções públicas sem autorização.
- Não gerar código sem testes correspondentes para as novas funcionalidades.
- Não inserir código duplicado ou sem necessidade.
- Não assumir comportamentos não especificados no contrato de negócio.

4 ARQUITETURA DO REPOSITÓRIO E PREPARAÇÃO PARA GIT/GITHUB

Desenho da Árvore do Repositório

Estrutura de arquivos que será versionada no GitHub na **Fase 2**:

```
sdd-task-analyzer/
├── README.md                    # Visão geral do projeto no GitHub
├── CONTEXT_RULES.md             # Arquivo de regras persistentes para a IA
├── specs/
│   └── task_analyzer_spec.md    # Especificação SDD (baseada nesta Fase 1)
├── tests/
│   └── test_harness.py          # Testes automatizados de validação
└── src/
    └── task_analyzer.py         # Código gerado via IA e homologado
```

Plano de Homologação Humana

Na Fase 2, todo código gerado pela IA será submetido a uma **revisão crítica humana** antes de ser aceito e versionado. A homologação seguirá os seguintes passos: (1) execução completa dos testes automatizados para validação funcional; (2) revisão de conformidade com o contrato de negócio e as CONTEXT_RULES; (3) análise de qualidade de código, legibilidade, tratamento de erros e aderência aos padrões definidos; (4) testes manuais complementares em casos de borda; e (5) somente após a aprovação, o código será commitado no repositório. Esse processo garante que a IA seja uma ferramenta produtiva, mas que a decisão final e a **responsabilidade técnica permaneçam humanas**.

IMPORTANTE: Estas seções (**3** e **4**) serão a base para a governança do desenvolvimento assistido por IA, garantindo consistência, rastreabilidade e qualidade em todas as etapas do projeto **TaskAnalyzer**.

REQUISITOS OBRIGATÓRIOS DO VÍDEO DE APRESENTAÇÃO

Gravar um vídeo simples explicando a sua especificação técnica com as seguintes regras:

APRESENTAÇÃO EM CÂMERA:

O estudante deve aparecer **obrigatoriamente** na câmera (mostrando o rosto e com áudio claro) enquanto apresenta seu documento.

DURAÇÃO:

5 minutos.

FORMATO PROIBIDO:

NÃO pode ser enviado como YouTube Shorts (deve ser gravado na **horizontal**).

PLATAFORMA:

Publicado no **YouTube** como "Público" ou "**Não Listado**".

PÚBLICO OU NÃO LISTADO

ENTREGA:

Inserir o **link ativo** do vídeo no Google Docs e enviar o arquivo **PDF** no **AVA**.

PDF

ROTEIRO DO VÍDEO

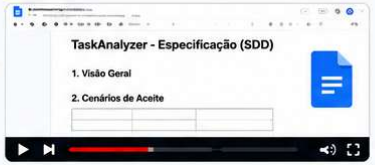
1 APRESENTAR-SE

Apresentar-se (nome, curso e motivação no Bootcamp III).



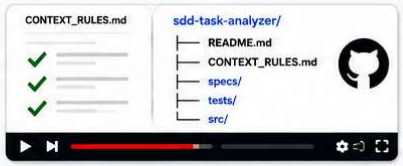
2 MOSTRAR A ESPECIFICAÇÃO

Mostrar a especificação do TaskAnalyzer no Google Docs e os cenários de aceite.



3 EXPLICAR REGRAS E ESTRUTURA

Explicar as regras de contexto criadas para guiar a IA e a estrutura planejada para o repositório no GitHub.



VISÃO GERAL DA CONTINUIDADE (PROJETO CUMULATIVO)

FASE 1 (Esta entrega - 10 pts)

Redação da especificação executável (SDD) no **Google Docs**, cenários de aceite, regras de contexto para IA, estrutura para o **GitHub** e vídeo de **5 minutos**.

FASE 2 (Entrega Intermediária - 15 pts)

Geração do código via assistentes de IA com base na especificação, execução do Test Harness (pytest), versionamento via Git e criação de Pull Requests no GitHub.

FASE 3 (Entrega Final - 25 pts)

Automação com **GitHub Actions (CI/CD)**, Relatório de Governança/Segurança (alucinações e ética) e homologação final do projeto.

IMPORTANTE: Siga todas as regras acima. A **clareza** da sua apresentação e da sua especificação é essencial para o sucesso do projeto!

CRITÉRIOS DE AVALIAÇÃO
(BAREMA DE CORREÇÃO)

CRITÉRIO	DESCRIÇÃO DETALHADA	PONTUAÇÃO
Estrutura e Conteúdo do Documento	Presença de todas as seções no Google Docs (Contrato de Negócio, Cenários de Aceite e Governança de IA), com clareza, rigor técnico e boa formatação (títulos, listas e tabelas).	**3,0 pts**
Especificação SDD e Preparação para Git/GitHub	Coerência no detalhamento das regras de contexto para a IA, lógica nos cenários de teste (Test Harness) e desenho coerente da estrutura do repositório para a Fase 2.	**3,0 pts**
Apresentação em Vídeo, Requisitos Técnicos e Formatação	Gravação com **participação obrigatória do estudante em câmera e áudio**, boa comunicação e clareza na explicação do projeto. Cumprimento dos requisitos técnicos: duração de **5 minutos**, publicado no YouTube em formato padrão horizontal (**proibido uso de Shorts**), link ativo inserido no documento e entrega realizada estritamente em **arquivo PDF no AVA**.	**4,0 pts**

⚖ **CÓDIGO DE ÉTICA DA EaD:** Mantenha os princípios de autoria individual na elaboração do trabalho e gravação do vídeo.

SDD – Spec-Driven Development

Desenvolvimento Orientado por Especificação

O novo paradigma da Engenharia de Software na era da Inteligência Artificial

O que é o SDD?

O **Spec-Driven Development (SDD)** é uma abordagem de desenvolvimento em que a **especificação técnica rígida e executável** guia todo o ciclo de construção do sistema, servindo como **contrato** entre humanos e agentes de **Inteligência Artificial**.

Principais Benefícios

- Reduz ambiguidades e retrabalho
- Diminui "alucinações" da IA
- Aumenta a qualidade e confiabilidade do código
- Facilita testes automatizados e integração contínua
- Melhora a rastreabilidade e manutenção do sistema

Por que o SDD é essencial agora?

- A IA já escreve código, mas precisa de contexto claro
- Especificações bem definidas geram melhores resultados
- Organiza o conhecimento do projeto de forma estruturada
- Eleva o papel do desenvolvedor para arquiteto e revisor crítico
- Integra desenvolvimento, testes, segurança e governança

Como funciona? (Ciclo do SDD)

1. **Especificação** – Definição detalhada de requisitos, regras e critérios de aceite.
2. **Contexto** – Regras e diretrizes claras para orientar os agentes de IA (CONTEXT_RULES).
3. **Geração** – Código gerado por IA com base na especificação e nas regras de contexto.
4. **Validação** – Execução de testes automatizados (Test Harness) e revisão crítica humana.
5. **Evolução** – Integração contínua (CI/CD), monitoramento e melhorias incrementais.

Componentes-chave do SDD

Especificação Executável
- Contrato de negócio
- Entradas e saídas bem definidas
- Regras de validação
- Cenários de aceite

Governança de Contexto
- Diretrizes arquiteturais
- Padrões de código
- Restrições e proibições
- Critérios de qualidade

Test Harness
- Cenários Dado-Quando-Então
- Testes automatizados (pytest)
- Validação objetiva do código
- Detecção precoce de erros

Versionamento
- Repositório no GitHub
- Histórico de mudanças
- Pull Requests
- Colaboração e rastreabilidade

SDD x Desenvolvimento Tradicional

Desenvolvimento Tradicional	SDD (Spec-Driven)
Requisitos dispersos e informais	Especificação clara e executável
Maior risco de desalinhamentos	Contexto controlado para IA
Testes definidos tardiamente	Testes desde o início
Código menos previsível	Código mais confiável e consistente
Documentação opcional	Documentação como contrato

O novo papel do desenvolvedor

Com o SDD, o desenvolvedor deixa de ser apenas um escritor de código e passa a atuar como:

- **Arquiteto de Especificações**
- **Curador de Contexto**
- **Homologador Crítico**

> *"No SDD, a especificação é o código-fonte da verdade, e a IA é a ferramenta de execução."*

Mais qualidade, segurança e produtividade para o desenvolvimento de software moderno.

Em resumo

- SDD = Especificação + Contexto + Testes + Governança
- Menos erros, mais qualidade e previsibilidade
- Base para projetos robustos e escaláveis
- O futuro do desenvolvimento com Inteligência Artificial

SDD – SPEC-DRIVEN DEVELOPMENT

Desenvolvimento Orientado por Especificações

O SDD é um paradigma moderno que coloca a **ESPECIFICAÇÃO TÉCNICA** como ponto de partida do desenvolvimento. Em vez de começar pelo código, começamos pelo **CONTRATO EXECUTÁVEL** que define o que o sistema deve fazer, como deve se comportar e como será validado.

IDEIA CENTRAL

Especificar primeiro, automatizar a validação e só depois implementar.

Especificação → Testes → Código → Validação

POR QUE SDD?

- **Alinhamento claro:** reduz ambiguidades entre negócio, desenvolvedores e IA.
- **Qualidade desde o início:** regras, restrições e cenários de teste definem o comportamento esperado.
- **IA com contexto:** a IA recebe regras e limites claros, evitando "alucinações" e saídas incorretas.
- **Facilita manutenção:** código gerado com base em uma especificação sólida é mais coeso e sustentável.
- **Rastreabilidade:** cada requisito tem um teste associado e um resultado verificável.

COMO FUNCIONA O SDD?

1. ESPECIFICAR	2. TESTAR (ANTES)	3. IMPLEMENTAR	4. VALIDAR	5. HOMOLOGAR
Definir requisitos, regras de negócio, entradas, saídas, restrições e cenários de aceite.	Transformar os cenários em testes automatizados (Test Harness).	Gerar o código com base na especificação e nas regras de contexto (IA ou dev).	Executar os testes automatizados para validar se o código atende ao contrato.	Revisão humana crítica do código, garantindo qualidade, segurança e aderência ética.

SDD transforma a especificação em um **CONTRATO EXECUTÁVEL**, validado por testes automatizados e revisado por humanos.

SDD vs. ABORDAGEM TRADICIONAL

TRADICIONAL (Code-First)	SDD (Spec-Driven)
Começa pelo código	Começa pela especificação
Requisitos muitas vezes implícitos ou instáveis	Contrato claro e estável
Testes escritos depois	Testes definidos antes da implementação
Mais retrabalho e bugs	Menos retrabalho, mais qualidade
Alinhamento difícil entre times	Alinhamento e comunicação efetivos

VS.

Resultado: entregas mais previsíveis, seguras e aderentes ao que o negócio realmente precisa.

OS 4 PILARES DO SDD

1. ESPECIFICAÇÃO

Requisitos funcionais e não funcionais, regras de negócio, contrato de interface, tipos de dados e restrições.

2. CENÁRIOS DE ACEITE

Cenários Dado–Quando–Então que descrevem o comportamento esperado em diferentes situações.

3. REGRAS DE CONTEXTO (CONTEXT_RULES)

Diretrizes arquiteturais e proibições explícitas para guiar IAs e desenvolvedores, garantindo consistência e segurança.

4. ESTRUTURA E GOVERNANÇA

Definição da estrutura do repositório, versionamento, revisão humana e fluxo de homologação.

EXEMPLO PRÁTICO – TASKANALYZER

No TaskAnalyzer, a especificação define que o sistema deve calcular métricas de produtividade a partir de tarefas.

ENTRADAS

Lista de tarefas com: id, titulo, prioridade, **data_criacao, data_conclusao, status.**

SAÍDAS (MÉTRICAS)

Tempo médio de conclusão, taxa de atraso, indicadores por prioridade, totais gerais.

RESTRIÇÕES DE NEGÓCIO

- Considerar apenas tarefas concluídas.
- Datas devem ser válidas e consistentes.
- Evitar divisão por zero.

CENÁRIOS DE ACEITE

Ex.: calcular métricas com dados válidos; tratar entradas inválidas com exceções específicas.

BENEFÍCIOS NA ERA DA IA

- **Menos "alucinações"** — A IA trabalha dentro de regras claras e limites bem definidos.
- **Código mais confiável** — Testes automatizados garantem que o código cumpra o contrato.
- **Aderência ao negócio** — A especificação mantém o foco no valor para o usuário.
- **Escalabilidade e reuso** — Padrões e estrutura facilitam evolução e reutilização.
- **Transparência e auditoria** — Regras e decisões documentadas e rastreáveis.
- **Governança e ética** — Revisão humana assegura responsabilidade e segurança.

BOAS PRÁTICAS PARA UM SDD EFICAZ

- **Seja claro e objetivo** — Use linguagem precisa e sem ambiguidades.
- **Especifique o essencial** — Foque no comportamento, não na solução.
- **Teste antes de codar** — Se o teste não consegue escrever, o requisito não está claro.
- **Documente tudo** — Especificações, decisões e regras devem estar registradas.
- **Revise com critério** — A validação humana é indispensável para qualidade e ética.

RESUMO

SDD é mais do que documentar: é criar um **contrato executável** que guia o desenvolvimento assistido por IA e humanos. Ele aumenta a **previsibilidade**, reduz riscos e garante que o produto final entregue valor real com qualidade, segurança e responsabilidade.

> "Especifique com rigor. Teste com confiança. Implemente com inteligência. Entregue com valor."

SDD – SPEC-DRIVEN DEVELOPMENT

Desenvolvimento Orientado por Especificação

No SDD, a especificação técnica é o ponto de partida e o contrato que orienta a geração do código por agentes de IA, garantindo alinhamento, qualidade e rastreabilidade durante todo o ciclo.

IDEIA CENTRAL

Especificar antes de codificar.
A especificação clara elimina ambiguidades, orienta a IA, facilita testes automatizados e permite validação humana crítica.

1 VISÃO GERAL E CONTRATO DE NEGÓCIO (SDD)

1.1 IDENTIFICAÇÃO

- Nome completo do estudante
- Curso, Polo/Turma
- E-mail institucional
- Data da elaboração do documento
- Versão da especificação

1.2 PROPÓSITO DO MÓDULO TASKANALYZER

- Analisar tarefas executadas e gerar métricas de produtividade.
- Calcular tempo médio de conclusão por tipo/prioridade.
- Identificar taxa de atraso (tarefas concluídas após o prazo).
- Gerar indicadores por prioridade (baixa, média, alta).
- Fornecer métricas para tomada de decisão e melhoria contínua.

1.3 CONTRATO EXECUTÁVEL DE INTERFACE

1.3.1 Entradas (Inputs)

Campo	Tipo	Obrigatório	Descrição / Restrições
id_tarefa	int	Sim	Identificador único da tarefa (> 0)
data_criacao	datetime	Sim	Data/hora de criação (UTC)
data_inicio	datetime	Não	Data/hora de início da execução
data_conclusao	datetime	Não	Data/hora de conclusão da execução
prazo	datetime	Sim	Data/hora limite para conclusão
prioridade	str enum	Sim	Valores: [baixa, média, alta]
status	str enum	Sim	Valores: [concluída, pendente, cancelada]

1.3.2 Saídas (Outputs)

Campo	Tipo	Descrição
tempo_medio_conclusao_min	float	Tempo médio de conclusão em minutos (por prioridade e geral)
taxa_atraso_percentual	float	Percentual de tarefas concluídas após o prazo (por prioridade e geral)
quantidade_tarefas	int	Total de tarefas consideradas na análise (por prioridade e geral)
indicadores_por_prioridade	dict	Estrutura com métricas agrupadas por prioridade (baixa, média, alta)

1.3.3 Regras de Negócio e Restrições

- Considerar apenas tarefas com status = 'concluída' para cálculo de tempo e atraso.
- Tempo de conclusão = data_conclusao - data_inicio (em minutos).
- Tarefa em atraso quando data_conclusao > prazo.
- Ignorar tarefas com datas inconsistentes (ex.: conclusão antes do início).
- Divisão por zero deve ser tratada quando não houver tarefas concluídas.
- Fuso horário sempre UTC.
- Arredondamentos: 2 casas decimais para tempos e percentuais.

CENÁRIOS DE ACEITE E TEST HARNESS

2.1 CENÁRIOS DE ACEITE (DADO – QUANDO – ENTÃO)

Cenário 1 – Sucesso (Métricas corretas)

Dado	um conjunto de tarefas válidas com diferentes prioridades, prazos e status (comcluidas e pendentes)
Quando	o analisador de tarefas for executado
Então	as métricas retornadas devem estar corretas, incluindo: • tempo médio de conclusão por prioridade e geral • taxa de atraso correta por prioridade e geral • quantidade de tarefas correta por prioridade

Cenário 2 – Exceção / Erro (Entradas inválidas)

Dado	uma entrada com datas inválidas (ex.: conclusão antes do início) ou ausência de tarefas concluídas (divisão por zero)
Quando	o analisador de tarefas for executado
Então	deve disparar uma exceção específica tratada, com mensagem clara e objetiva ao usuário

2.2 PLANEJAMENTO DO TEST HARNESS

Cenários de Aceite (documento SDD)	• Descrevem o comportamento esperado do sistema • Servem como referência para os testes
Casos de Teste (derivados dos cenários)	• Decompor cada cenário em casos específicos • Definir dados de entrada e resultados esperados
Testes Automatizados (pytest)	• Implementar testes com pytest • Usar fixtures para preparar dados • Validar outputs, tipos e exceções
Execução e Validação (contínua)	• Executar os testes automaticamente • Qualquer falha bloqueia o avanço para a próxima fase
Relatórios e Evidências (resultados)	• Gerar relatórios de cobertura e resultados • Garantir rastreabilidade para cada cenário

Objetivo: garantir que o código gerado por IA esteja correto, conforme o contrato e os cenários definidos.

GOVERNANÇA DE CONTEXTO E REGRAS PARA IA (CONTEXT_RULES)

3.1 DIRETRIZES ARQUITETURAIS (OBRIGATÓRIAS)

- Linguagem: Python 3.11 ou superior
- Usar type hints (anotações de tipo) em todo o código
- Seguir PEP 8 e boas práticas de código limpo
- Funções pequenas, coesas e com responsabilidade única
- Documentação docstring em padrão Google
- Logs estruturados com o módulo **logging**
- Tratamento de exceções específico e mensagens claras
- **Código legível**, manutenível e com nomes descritivos
- Separar responsabilidades (SRP) e evitar lógica em scripts

3.2 PROIBIÇÕES EXPLÍCITAS (REGRAS RESTRITIVAS)

- Não utilizar bibliotecas externas não autorizadas.
- Não alterar os cenários de teste fornecidos.
- Não modificar a estrutura de pastas definida.
- Não persistir dados em arquivos ou bancos de dados.
- Não alterar assinatura (nome e parâmetros) das funções públicas.
- Não gerar código sem testes correspondentes.
- Não inserir código duplicado ou dennecessário.
- Não assumir comportamentos não especificados.
- Não comentar código em excesso (apenas o necessário).
- Não retornar valores diferentes do contrato definido.

3.3 REGRAS DE INTERAÇÃO COM IA

- Fornecer sempre o contexto completo antes da solicitação.
- Validar se a IA compreendeu o contrato antes de gerar código.
- Solicitar explicações quando houver dúvidas na solução.
- Revisar criticamente todo código gerado.
- Rejeitar respostas que violem as proibições ou o contrato.

Essas regras compõem o arquivo CONTEXT_RULES.md, que será a fonte de verdade para os agentes de IA.

ARQUITETURA DO REPOSITÓRIO E PREPARAÇÃO PARA GIT/GITHUB

4.1 ÁRVORE DO REPOSITÓRIO (FASE 2)

```
sdd-task-analyzer/
├── README.md                    # Visão geral do projeto
├── CONTEXT_RULES.md             # Regras persistentes para IA
├── specs/
│   └── task_analyzer_spec.md    # Especificação SDD)
├── tests/
│   └── test_harness.py          # Testes automatizados (pytest)
├── src/
│   └── task_analyzer.py         # Código gerado por IA
├── .gitignore                   # Arquivos e pastas ignoradas
└── requirements.txt             # Dependências autorizadas
```

4.2 PLANO DE HOMOLOGAÇÃO HUMANA

Todo código gerado pela IA será submetido a uma revisão crítica humana antes de ser aceito e versionado. A homologação seguirá os seguintes passos: (1) executar os testes automatizados e verificar cobertura; (2) revisar conformidade com o contrato de negócio e regras de contexto; (3) analisar qualidade, legibilidade e tratamento de erros; (4) validar se não há violação das proibições; (5) realizar testes manuais complementares quando necessário; (6) só então aprovar e fazer commit no repositório.

4.3 ESTRATÉGIA DE VERSIONAMENTO (FASE 2)

- Git Flow simplificado: main, develop, feature/*
- Commits pequenos e descritivos
- Pull Requests para revisão e aprovação
- Revisões obrigatórias antes do merge
- Tags para releases importantes

Estrutura e processo preparados para colaboração, rastreabilidade e integração contínua na Fase 3.

VISÃO GERAL DA CONTINUIDADE (PROJETO CUMULATIVO)

FASE 1 – ESTA ENTREGA (10 pts)

- Especificação executável (SDD) no Google Docs
- Cenários de aceite e Test Harness (planejado)
- Regras de contexto para IA (CONTEXT_RULES)
- Estrutura para Git/GitHub
- Vídeo de 5 minutos explicativo

FASE 2 – ENTREGA INTERMEDIÁRIA (15 pts)

- Geração do código via IA com base na SDD
- Implementação do Test Harness (pytest)
- Execução e validação dos testes
- Versionamento com Git
- Pull Requests no GitHub

FASE 3 – ENTREGA FINAL (25 pts)

- Automação com GitHub Actions (CI/CD)
- Relatório de Governança e Segurança
- Análise de alucinações e ética
- Homologação final e deploy (se aplicável)
- Documentação final do projeto

ENTREGÁVEIS DA FASE 1

- Documento Google Docs completo (especificação, cenários, regras e estrutura)
- Vídeo de apresentação (5 min, horizontal, YouTube – público ou não listado)
- PDF exportado contendo:
  - Link do Google Docs
  - Link do vídeo no YouTube
  - Identificação do estudante

BOAS PRÁTICAS SDD

- Especificar com clareza
- Contextualizar para a IA
- Testar continuamente
- Revisar criticamente
- Documentar sempre

LEMBRE-SE: No SDD, a especificação é o contrato. A IA é a executora. Você é o arquiteto e o homologador.

IA HARDNESS

Robustez, Confiabilidade e Segurança na Era da Inteligência Artificial

Construindo sistemas de IA resilientes, éticos e confiáveis

O QUE É IA HARDNESS?

IA Hardness é o conjunto de práticas, técnicas e políticas que tornam sistemas de Inteligência Artificial robustos, seguros, confiáveis, explicáveis e alinhados a princípios éticos, reduzindo riscos e aumentando a resiliência em todo o ciclo de vida da IA.

- Reduz riscos e falhas
- Aumenta a confiabilidade
- Protege dados e usuários
- Promove ética e transparência
- Garante conformidade e governança

POR QUE IA HARDNESS É ESSENCIAL?

- Modelos podem falhar, ser manipulados ou gerar resultados tendenciosos.
- Dados podem estar incompletos, enviesados ou vazados.
- Ataques adversariais e uso indevido são reais e crescentes.
- Sistemas de IA impactam pessoas e decisões críticas.
- Regulamentações (ex.: LGPD, AI Act, ISO/IEC 42001) exigem responsabilidade e rastreabilidade.

ONDE SE APLICA?

Assistentes virtuais · Visão computacional · Recomendação e personalização · Análise preditiva e decisão · IA generativa e copilots · Robótica e automação

CICLO DE VIDA DA IA COM HARDNESS

1. Planejamento e Requisitos
2. Dados
3. Desenvolvimento do Modelo
4. Testes e Validação
5. Implantação (Deploy)
6. Operação e Monitoramento
7. Melhoria Contínua

PRINCÍPIOS FUNDAMENTAIS

- **Confiabilidade:** resultados consistentes e corretos.
- **Segurança:** proteção contra ameaças e vazamentos.
- **Privacidade:** uso ético e responsável de dados.
- **Transparência:** decisões explicáveis e auditáveis.
- **Equidade:** prevenção de vieses e discriminações.
- **Responsabilidade:** pessoas e processos respondem pelos impactos da IA.

PILARES DA IA HARDNESS

1. QUALIDADE E GOVERNANÇA DE DADOS

- Coleta ética e consentida de dados.
- Dados representativos, completos, consistentes e atualizados.
- Detecção e mitigação de vieses nos dados.
- Anonimização e minimização (LGPD).
- Catálogo de dados e linhagem (data lineage).

Dados de qualidade são a base de uma IA confiável.

2. ROBUSTEZ E DESEMPENHO DO MODELO

- Validação cruzada e métricas adequadas.
- Testes de robustez e generalização.
- Tratamento de overfitting/underfitting.
- Calibração de probabilidades.
- Definição de limites de confiança.

Modelos devem funcionar bem em diferentes condições reais.

3. SEGURANÇA E PROTEÇÃO CONTRA AMEAÇAS

- Defesa contra ataques adversariais.
- Validação de entradas (input validation).
- Proteção contra extração de modelo.
- Red team e testes de penetração.
- Gerenciamento de vulnerabilidades.

Antecipe ataques e reduza superfícies de risco.

4. EXPLICABILIDADE E TRANSPARÊNCIA

- Modelos interpretáveis quando possível.
- Explicações para decisões importantes.
- Documentação clara de dados, features e lógica de decisão.
- Comunicação compreensível aos usuários.

Decisões explicáveis geram confiança e permitem auditoria.

5. MONITORAMENTO E CONFIABILIDADE OPERACIONAL

- Monitoramento contínuo de desempenho.
- Detecção de drift de dados e de conceito.
- Alertas para anomalias e degradação.
- Logs, métricas e rastreabilidade.
- Planos de resposta e rollback.

IA em produção exige observabilidade e resposta rápida.

6. ÉTICA, VIÉS E IMPACTO HUMANO

- Avaliação de vieses e impactos sociais.
- Equidade entre grupos e populações.
- Princípio do humano no controle.
- Uso responsável e alinhado ao bem comum.
- Prestação de contas e auditoria ética.

IA deve ser justa, inclusiva e centrada nas pessoas.

RISCOS COMUNS (SEM IA HARDNESS)

- Previsões incorretas e decisões injustas.
- Vieses algorítmicos e discriminação.
- Vazamento de dados sensíveis.
- Ataques adversariais e manipulação de modelo.
- Falta de explicabilidade e transparência.
- Não conformidade legal e regulatória.
- Perda de confiança do usuário e reputação.

BOAS PRÁTICAS ESSENCIAIS

- Defina requisitos éticos e de segurança desde o início.
- Conheça seus dados profundamente.
- Documente tudo: dados, modelo, decisões e versões.
- Teste exaustivamente (funcional, adversarial e ético).
- Implemente explicabilidade por design.
- Monitore continuamente em produção.
- Tenha planos de contingência e governança clara.
- Capacite pessoas e promova cultura de responsabilidade.

PADRÕES E REFERÊNCIAS IMPORTANTES

- ISO/IEC 42001 – Sistema de Gestão de IA
- NIST AI RMF – Framework de Gestão de Riscos de IA
- OWASP Top 10 for ML – Principais riscos em ML
- ISO/IEC 23894 – Gestão de Riscos de IA
- EU AI Act – Regulamento de IA da União Europeia
- LGPD – Lei Geral de Proteção de Dados
- IEEE Ethically Aligned Design – Princípios éticos para IA

MÉTRICAS DE IA HARDNESS

- Acurácia, Precisão, Recall, F1-score
- Robustez (teste adversarial) (%)
- Drift de dados e de conceito
- Taxa de explicabilidade (cobertura)
- Incidentes de segurança detectados
- Tempo de resposta a incidentes (MTTR)
- Satisfação e confiança do usuário

CHECKLIST RÁPIDO DE IA HARDNESS

- [ ] Dados governados e com qualidade?
- [ ] Vieses avaliados e mitigados?
- [ ] Modelo testado para robustez?
- [ ] Explicações disponíveis e compreensíveis?
- [ ] Ameaças identificadas e mitigadas?
- [ ] Monitoramento ativo em produção?
- [ ] Conformidade legal e ética atendida?
- [ ] Documentação completa e atualizada?
- [ ] Plano de resposta a incidentes definido?

EM RESUMO

IA Hardness não é apenas sobre tecnologia, é sobre responsabilidade, confiança e impacto positivo. Sistemas de IA robustos e éticos constroem um futuro mais seguro, justo e sustentável para todos.

REGRAS DE OURO DA IA HARDNESS

- Qualidade dos dados é fundamental
- Teste, teste e teste (não confie apenas em métricas únicas)
- Explique ou não automatize decisões críticas
- Monitore sempre, aprenda sempre
- Coloque pessoas e ética no centro de todas as decisões
- Previna hoje os problemas de amanhã

LEMBRE-SE

IA Hardness não é um destino, é uma jornada contínua de melhoria e responsabilidade.

IA HARDNESS

Robustez, Confiabilidade e Segurança na Era da IA

Construindo sistemas de IA resilientes, éticos e confiáveis em todo o ciclo de vida

O QUE É IA HARDNESS?

Conjunto de práticas, técnicas e políticas que tornam sistemas de Inteligência Artificial robustos, seguros, confiáveis, explicáveis e alinhados a princípios éticos, reduzindo riscos e aumentando a resiliência em todo o ciclo de vida da IA.

- Reduz riscos e falhas
- Aumenta a confiabilidade
- Protege dados e usuários
- Promove ética e transparência
- Garante conformidade e governança

POR QUE IA HARDNESS É ESSENCIAL?

- Modelos podem falhar, ser manipulados ou gerar resultados tendenciosos.
- Dados podem estar incompletos, enviesados ou vazados.
- Ataques adversariais e uso indevido são reais e crescentes.
- Sistemas de IA impactam pessoas e decisões críticas.
- Regulamentações (ex.: LGPD, AI Act, ISO/IEC 42001) exigem responsabilidade e rastreabilidade.

ONDE SE APLICA?

- Assistentes virtuais
- Visão computacional
- Recomendação e personalização
- Análise preditiva e decisão
- IA generativa e copilots
- Robótica e automação

CICLO DE VIDA DA IA COM HARDNESS

1. Planejamento e Requisitos
2. Dados
3. Desenvolvimento do Modelo
4. Testes e Validação
5. Implantação (Deploy)
6. Operação e Monitoramento
7. Melhoria Contínua

PRINCÍPIOS FUNDAMENTAIS

- **Confiabilidade:** resultados consistentes e corretos.
- **Segurança:** proteção contra ameaças e vazamentos.
- **Privacidade:** uso ético e responsável de dados.
- **Transparência:** decisões explicáveis e auditáveis.
- **Equidade:** prevenção de vieses e discriminações.
- **Responsabilidade:** pessoas e processos respondem pelos impactos da IA.

PILARES DA IA HARDNESS – DESDOBRAMENTO DETALHADO

1. QUALIDADE E GOVERNANÇA DE DADOS

Desdobramento

- Coleta ética e consentida de dados.
- Dados representativos, completos, consistentes e atualizados.
- Detecção e mitigação de vieses nos dados.
- Anonimização e minimização (LGPD).
- Catálogo de dados e linhagem (data lineage).

Boas práticas

- Perfilamento de dados
- Regras de qualidade e validações
- Versionamento de datasets
- Políticas de retenção e descarte

Resultado esperado

Dados de qualidade são a base de uma IA confiável.

2. ROBUSTEZ E DESEMPENHO DO MODELO

Desdobramento

- Validação cruzada e métricas adequadas.
- Testes de robustez e generalização.
- Tratamento de overfitting/underfitting.
- Calibração de probabilidades.
- Definição de limites de confiança.

Boas práticas

- Benchmarking e comparação de modelos
- Técnicas de regularização
- Stress tests com dados adversariais
- Documentação de suposições e limites

Resultado esperado

Modelos funcionam bem em diferentes condições reais.

3. SEGURANÇA E PROTEÇÃO CONTRA AMEAÇAS

Desdobramento

- Defesa contra ataques adversariais.
- Validação de entradas (input validation).
- Proteção contra extração de modelo.
- Red team e testes de penetração.
- Gerenciamento de vulnerabilidades.

Boas práticas

- Threat modeling específico para IA
- Rate limiting e controle de acesso
- Monitoramento de comportamentos anômalos
- Atualizações e patches de segurança

Resultado esperado

Sistemas protegidos contra ameaças e uso indevido.

4. EXPLICABILIDADE E TRANSPARÊNCIA

Desdobramento

- Modelos interpretáveis quando possível.
- Explicações para decisões importantes.
- Documentação clara de dados, features e lógica de decisão.
- Comunicação compreensível aos usuários.

Boas práticas

- XAI: SHAP, LIME, PDP, Feature Importance
- Relatórios de explicabilidade
- Dashboards e visualizações
- Registro das decisões (audit trail)

Resultado esperado

Decisões explicáveis geram confiança e permitem auditoria.

5. MONITORAMENTO E CONFIABILIDADE OPERACIONAL

Desdobramento

- Monitoramento contínuo de desempenho.
- Detecção de drift de dados e de conceito.
- Alertas para anomalias e degradação.
- Logs, métricas e rastreabilidade.
- Planos de resposta e rollback.

Boas práticas

- MLOps e observabilidade
- SLA/SLO e indicadores de saúde
- Canary/Shadow Deploy
- Testes em produção (A/B, holdout)

Resultado esperado

IA em produção exige observabilidade e resposta rápida.

6. ÉTICA, VIÉS E IMPACTO HUMANO

Desdobramento

- Avaliação de vieses e impactos sociais.
- Equidade entre grupos e populações.
- Princípio do humano no controle.
- Uso responsável e alinhado ao bem comum.
- Prestação de contas e auditoria ética.

Boas práticas

- Métricas de equidade (DP, EO, etc.)
- Revisão ética multidisciplinar
- Participação e feedback de usuários
- Transparência sobre limitações

Resultado esperado

IA deve ser justa, inclusiva e centrada nas pessoas.

RISCOS COMUNS (SEM IA HARDNESS)

- Previsões incorretas e decisões injustas.
- Vieses algorítmicos e discriminação.
- Vazamento de dados sensíveis.
- Ataques adversariais e manipulação de modelo.
- Falta de explicabilidade e transparência.
- Não conformidade legal e regulatória.
- Perda de confiança do usuário e reputação.

BOAS PRÁTICAS ESSENCIAIS (TRANSVERSAIS)

- Defina requisitos éticos e de segurança desde o início.
- Conheça seus dados profundamente.
- Documente tudo: dados, modelo, decisões e versões.
- Teste exaustivamente (funcional, adversarial e ético).

PADRÕES E REFERÊNCIAS IMPORTANTES

- ISO/IEC 42001 – Sistema de Gestão de IA
- NIST AI RMF – Framework de Gestão de Riscos de IA
- OWASP Top 10 for ML – Principais riscos em ML
- ISO/IEC 23894 – Gestão de Riscos de IA
- EU AI Act – Regulamento de IA da União Europeia
- LGPD – Lei Geral de Proteção de Dados
- IEEE Ethically Aligned Design – Princípios éticos para IA

MÉTRICAS DE IA HARDNESS (EXEMPLOS)

- Acurácia, Precisão, Recall, F1-score
- Robustez (teste adversarial) (%)
- Drift de dados e de conceito
- Taxa de explicabilidade (cobertura)
- Incidentes de segurança detectados
- Tempo de resposta a incidentes (MTTR)
- Satisfação e confiança do usuário

CHECKLIST RÁPIDO DE IA HARDNESS

- [ ] Dados governados e com qualidade?
- [ ] Vieses avaliados e mitigados?
- [ ] Modelo testado para robustez?
- [ ] Explicações disponíveis e compreensíveis?
- [ ] Ameaças identificadas e mitigadas?
- [ ] Monitoramento ativo em produção?
- [ ] Conformidade legal e ética atendida?
- [ ] Documentação completa e atualizada?
- [ ] Plano de resposta a incidentes definido?

EM RESUMO

IA Hardness não é apenas sobre tecnologia. É sobre responsabilidade, confiança e impacto positivo. Sistemas de IA robustos e éticos constroem um futuro mais seguro, justo e sustentável para todos.

REGRAS DE OURO DA IA HARDNESS

- Qualidade dos dados é fundamental
- Teste, teste e teste (não confie apenas em métricas únicas)
- Explique ou não automatize decisões críticas
- Monitore sempre, aprenda sempre
- Coloque pessoas e ética no centro de todas as decisões
- Previna hoje os problemas de amanhã

LEMBRE-SE

IA Hardness não é um destino, é uma jornada contínua de melhoria e responsabilidade.

IA HARDNESS = ROBUSTEZ TÉCNICA + SEGURANÇA + ÉTICA + GOVERNANÇA + CONFIANÇA

RESULTADO: SISTEMAS DE IA QUE ENTREGAM VALOR COM SEGURANÇA E RESPONSABILIDADE.